University Of Central Punjab

Faculty of Information Technology

PROJECT REPORT

Customer Churn Prediction

Using Supervised Machine Learning Models

Dataset: IBM Telco Customer Churn

Course / Lab:	Introduction to DataScience
Instructor:	Miss Zar Bakht Imtiaz
Section:	G-4
Submission Date:	January 15, 2026

Group Members

Student Name	Registration No.
Awais Arif	L1F22BSCS0398
Maliha Fajar	L1F22BSCS1028
Afaq Imran	L1F22BSCS0371
Hassan Zahid	L1F22BSCS0397
Mian Abdullah Nadeem	L1F22BSCS0385

Customer Churn Prediction Using Supervised Machine Learning Models

1. Abstract

This project predicts telecom customer churn using the Telco Customer Churn dataset. We built a completely supervised ML pipeline with preprocessing, EDA, and SMOTE–Tomek applied only on the training set to handle class imbalance. Five models (Random Forest, XGBoost, LightGBM, Logistic Regression, CatBoost) were trained, and a soft-voting ensemble combined their probabilities, and evaluated using accuracy, precision, recall, F1-score, and confusion matrix, robustness checked through bootstrapped confidence intervals and error analysis.

2. Introduction

Customer churn lowers recurring revenue, so early prediction helps companies take retention actions like offers, better support, and customized plans. Prediction is challenging because churners are usually a minority, and customer behaviors overlap. Since the dataset has both numeric (Monthly/Total Charges) and categorical features (Contract, Payment Method), a strong preprocessing pipeline is required. We compare multiple supervised models and use soft voting for more stable, balanced results, with the trade-off of higher complexity than a single interpretable model.

3. Problem Statement

Given a customer's attributes—such as **tenure**, **subscribed services**, **billing details etc**—the goal is to **predict whether the customer will churn** (leave telecom service) or **remain active**

- **Churn = 1** → Customer **left / cancelled** the service
- **Churn = 0** → Customer **stayed / continued** the service
  Since the target has **two possible outcomes (0 or 1)**, this is a **binary classification** problem, where the model learns patterns from historical customer data to classify new customers as **likely to churn** or **likely to stay**.

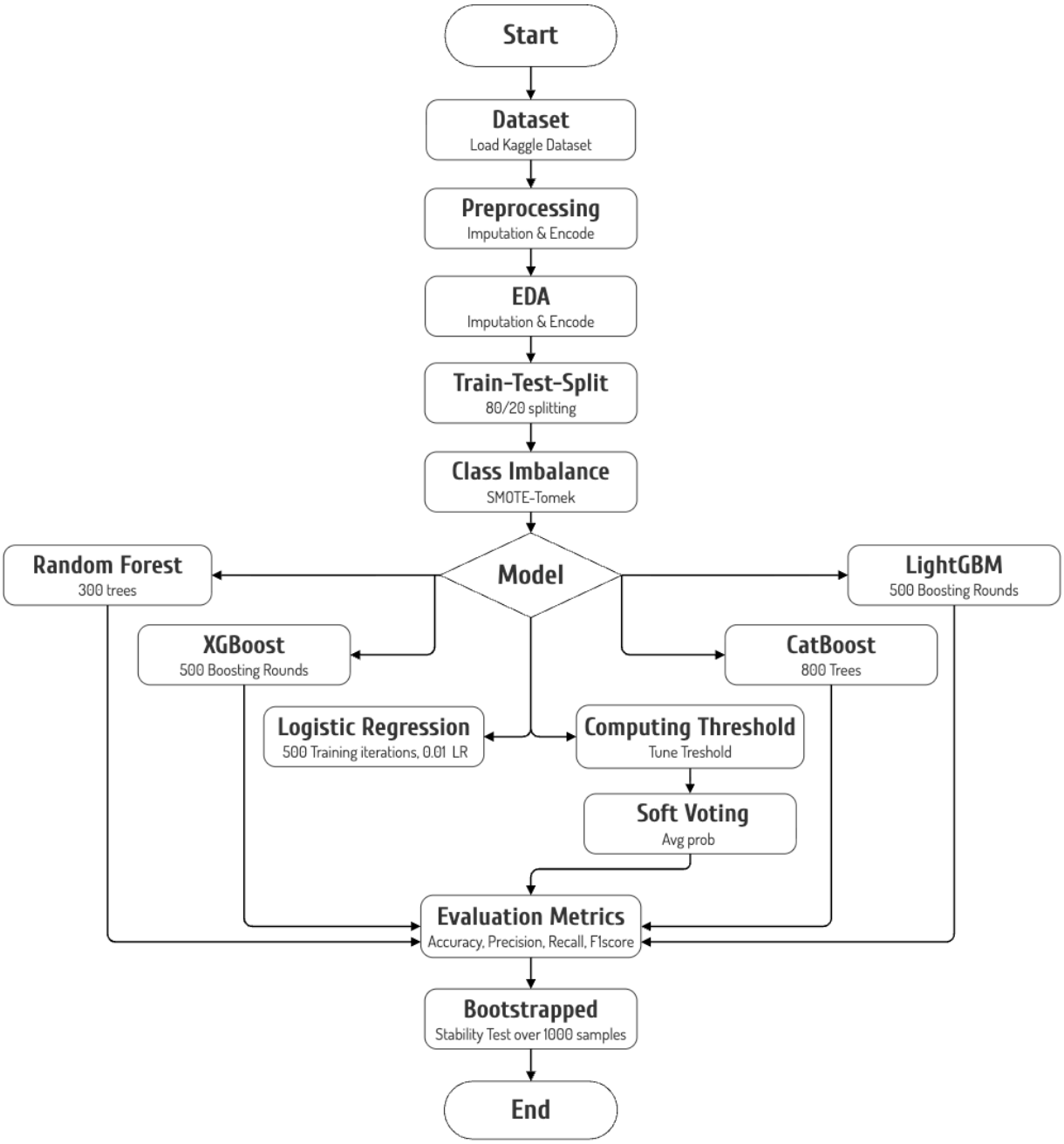
4. Dataset Overview

The dataset used in this project is the **Telco Customer Churn** dataset, which contains information for **7,043 customers** with **21 features** describing customer profiles The main objective is to predict the **target variable, Churn**, which shows whether a customer left the service or stayed. In the original dataset, churn is recorded as **"Yes"** or **"No"**, and for machine

learning purposes it is converted into a binary format where **Yes = 1 (customer churned)** and **No = 0 (customer retained)**.

5. Methodology

5.1 Workflow



5.2 Preprocessing

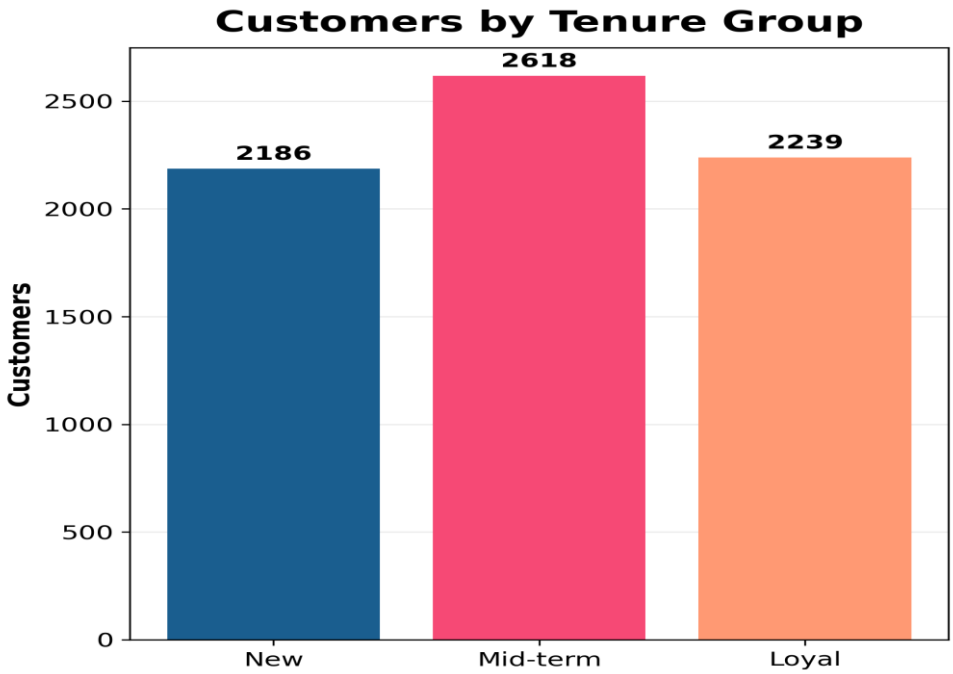
- Convert **TotalCharges** to numeric (handle blanks as NaN).
- Impute using mean.

- Remove duplicate rows (if any).
- Drop identifier column **customerID** because it does not carry predictive meaning.
- Encode Churn into binary labels (Yes→1, No→0). One Hot Encoding on categorical features.

5.3 Feature Engineering

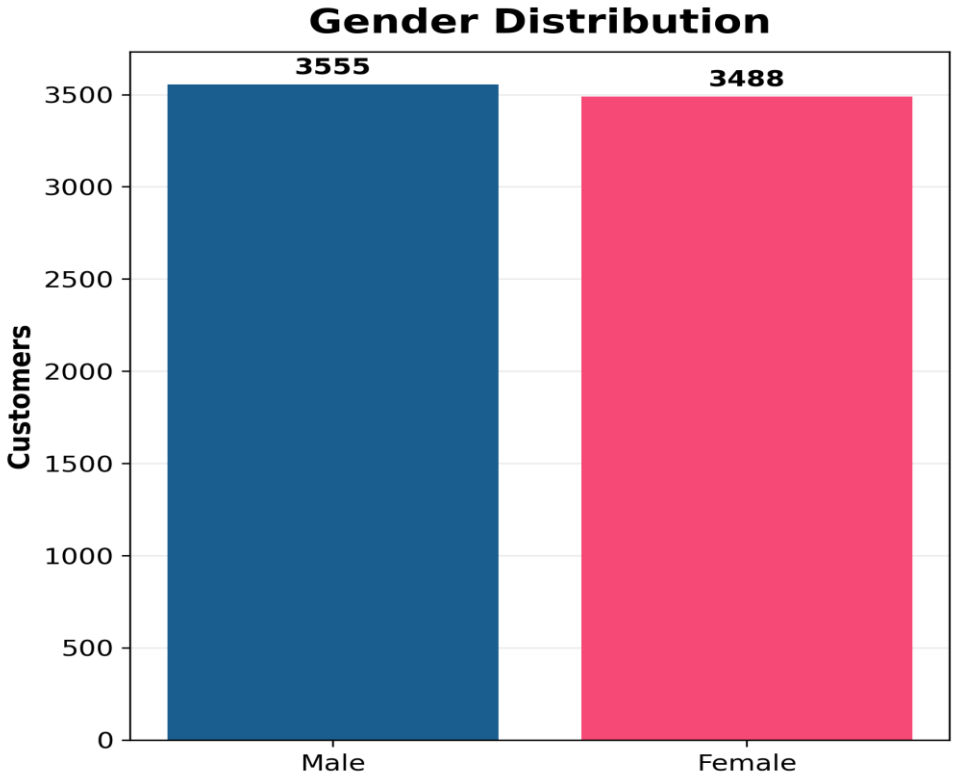
Created **tenure_group** binning of tenure.

- **New** → customers with low tenure.
- **Mid-term** → customers with medium tenure.
- **Loyal** → customers with high tenure

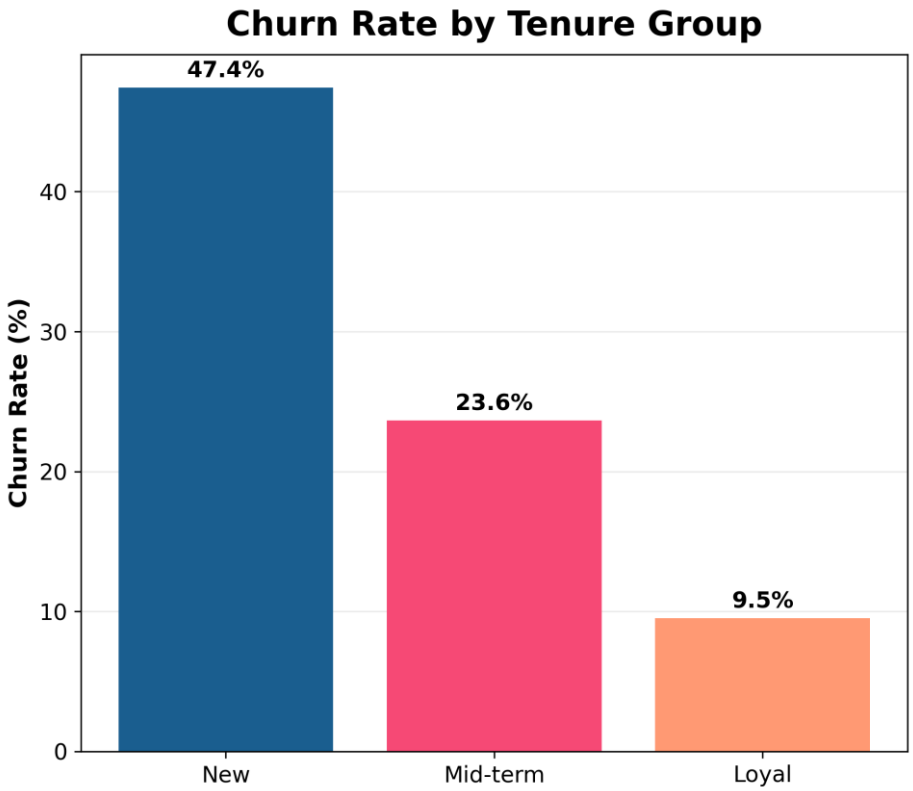


5.4 Exploratory Data Analysis (EDA)

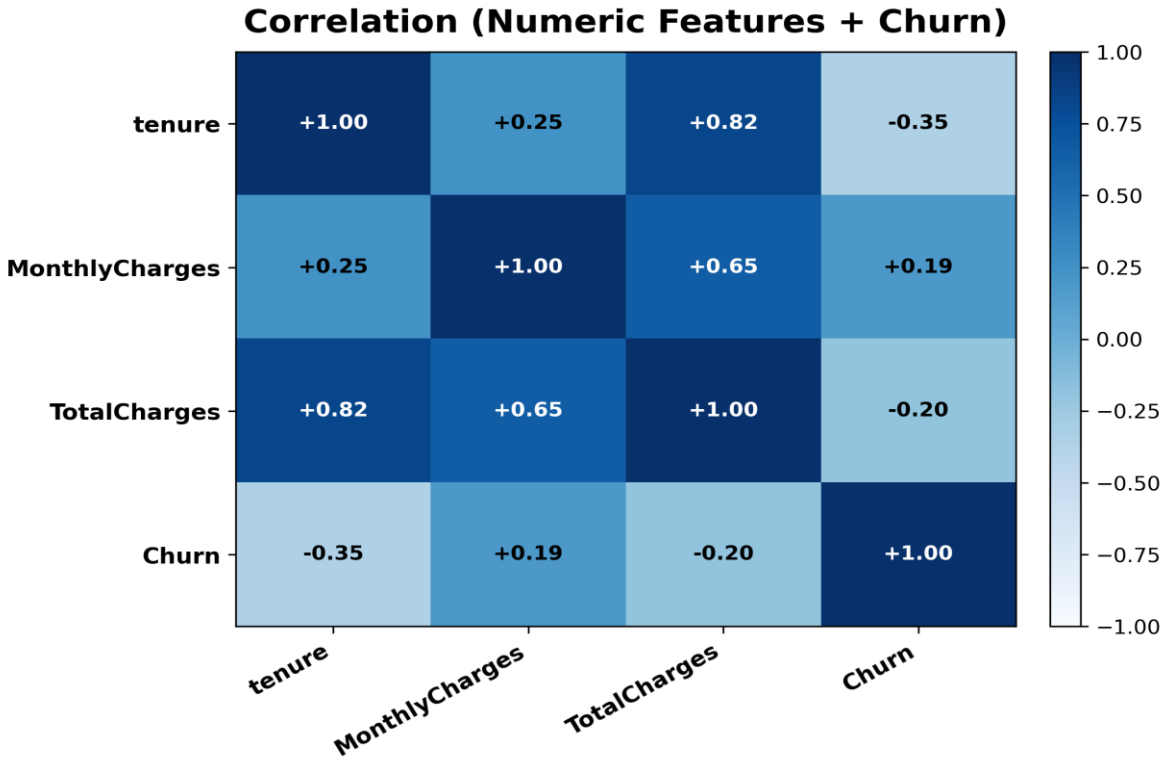
- **Check class distribution of males and females in dataset.**



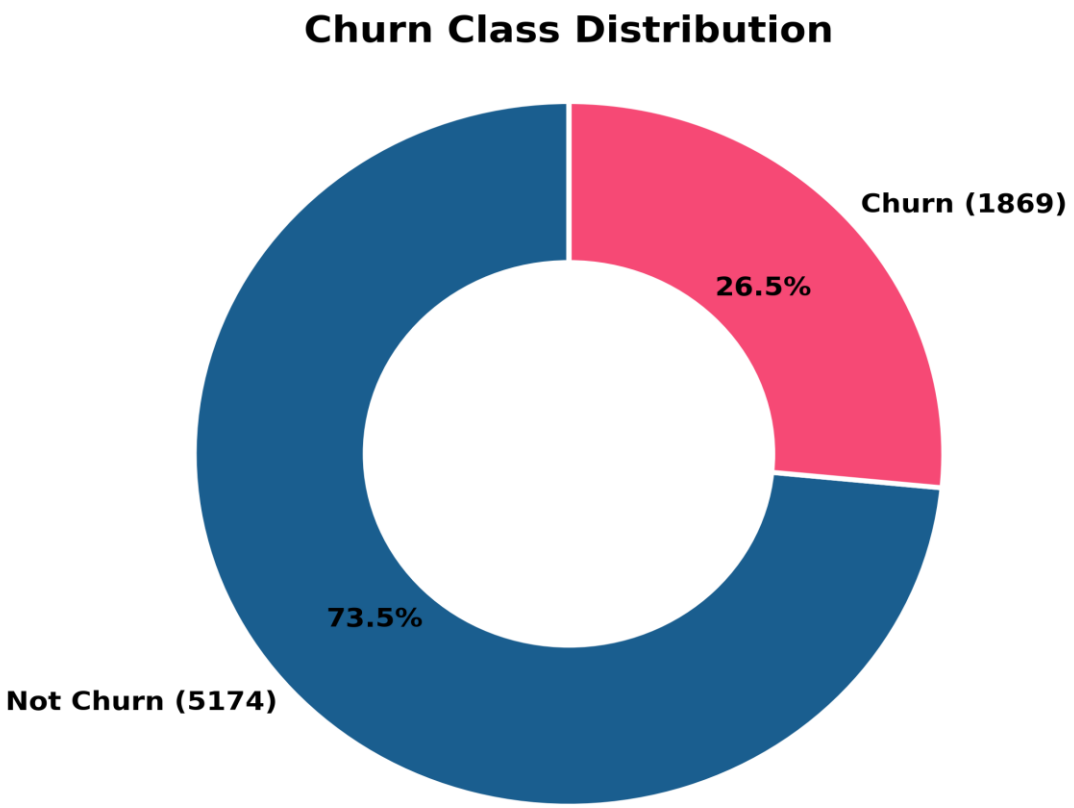
- **Analyze churn rate by tenure groups (New, Mid-term, Loyal) to understand churn patterns across customer lifecycle.**



- **Compute correlation between numeric features and churn to identify linear relationships and potential drivers.**



- **Check class distribution to confirm class imbalance between churn and non-churn customers.**



5.6 Train/Test Split

All models were trained using an **80/20 train–test split**.

- The **training set (80%)** was used for model training, model selection (including hyperparameter tuning), and any resampling techniques.
- The **test set (20%)** was kept separate and used **only for the final evaluation** to provide an unbiased estimate of model performance.

5.7 Class Imbalance Handling (SMOTE–Tomek)

Customer churn datasets typically have an **imbalanced class distribution**, where churners are the minority. This can bias models toward predicting the majority class (non-churn), reducing the ability to correctly detect churners.

To address this, we applied **SMOTE–Tomek** on the training data:

- **SMOTE (Synthetic Minority Over-sampling Technique)** creates **new synthetic churn samples** by interpolating minority-class neighbors, improving the model's ability to learn churn patterns and increasing sensitivity to churn cases.
- **Tomek Links** remove **ambiguous/noisy samples near the decision boundary**, improving class separability and reducing overlap between churn and non-churn customers.

5.8 Modeling Hyperparameters

Hyperparameters are the settings chosen before training a model that control how the algorithm learns (they are not learned from the data like weights/coefficients), such as the learning rate, number of trees/iterations, maximum depth, and regularization strength, and choosing them carefully can improve performance while reducing overfitting or underfitting. In this project, each model was trained with a defined set of hyperparameters, and **model selection/tuning was performed only on the training set** (to avoid test leakage).

Typical hyperparameters include:

Model	Key Hyperparameters	What it Represents
Random Forest	n_estimators=500	500 trees (more trees → usually better stability)
XGBoost	n_estimators=500, learning_rate=0.01	500 boosting rounds with learning rate to reduce overfitting.

LightGBM	n_estimators=500, learning_rate=0.01	500 boosting rounds with learning rate controls step size to reduce overfitting.
Logistic Regression	max_iter= 500, learning_rate = 0.01	500 iterations with learning rate 0.01
CatBoost	iterations=800, learning_rate=0.01, depth=6	800 boosting rounds with learning rate 0.01 and tree height 6

5.9 Soft Voting Ensemble

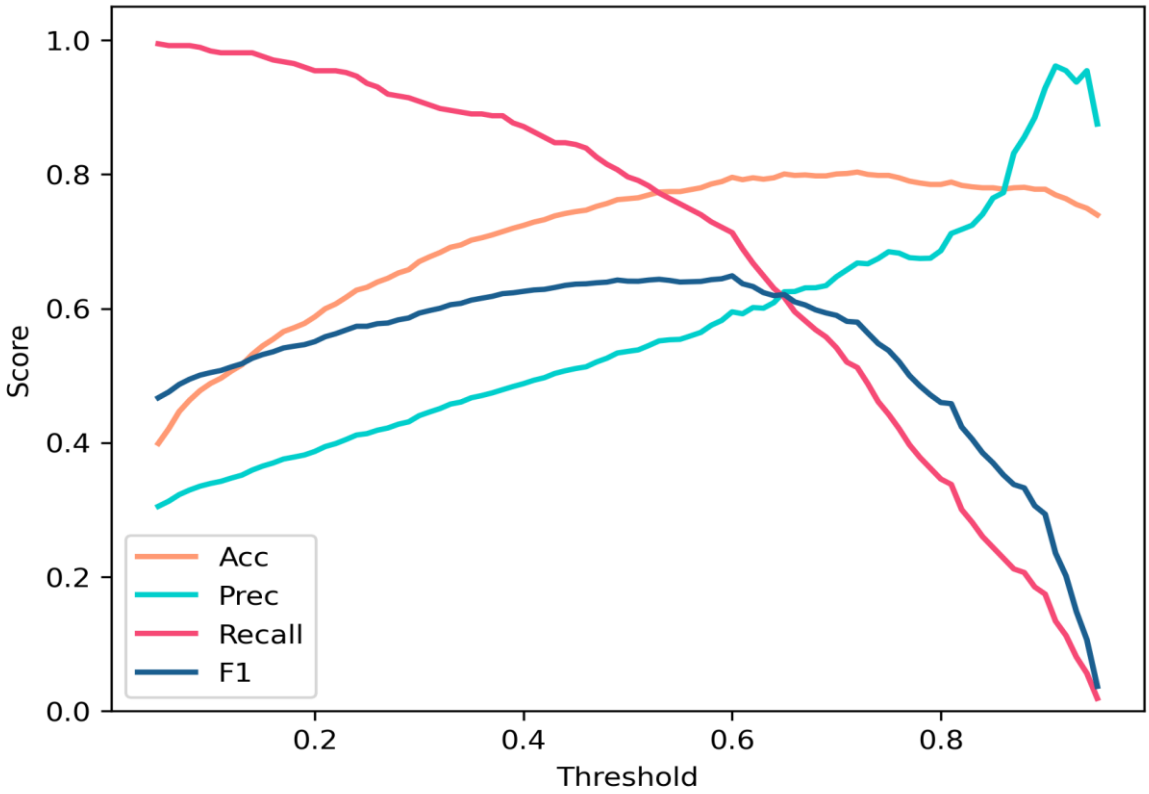
In a **soft voting ensemble**, each model predicts the **probability of churn** for a customer. We then **combine** these probabilities by taking their **average** to produce one final churn score. Using probabilities (instead of hard 0/1 votes) usually gives **more stable and balanced** predictions.This final score is then compared with the **chosen decision threshold** to generate the final class.

- **Final score:** average of all model probabilities
- **Prediction rule:**
  - if **p_final ≥ threshold** → **Churn = 1**
  - otherwise → **Churn = 0**

5.10 Threshold Tuning

The default decision threshold of **0.50** is not always optimal for **imbalanced datasets** like churn prediction. Changing the threshold helps control the trade-off between catching churners and avoiding false alarms.

- **Increasing the threshold** (e.g., above 0.50) usually **reduces false positives** (fewer non-churn customers predicted as churn), but it may **increase false negatives** (more churners missed).
- **Decreasing the threshold** improves **recall** (captures more churners), but it can **raise false positives** (more false alarms).
- After testing different values, the **best-performing threshold was 0.60**, as it provided the most **balanced results across key metrics** (accuracy, precision, recall, and F1-score).



6. Evaluation

Model performance was evaluated on the **held-out test set** using multiple complementary metrics to capture overall quality and the churn–non-churn trade-off.

- **Accuracy** measures overall correctness.
- **Precision** reflects the reliability of churn alarms.
- **Recall** measures how many actual churners are captured;
- **F1-score** provides a balanced summary of precision and recall.
- **Confusion matrix** to understand false positives and false negatives.

7. Results

7.1 Overall model performance

Model	Accuracy	Precision	Recall	F1
RF	0.7175	0.4809	0.8418	0.6121

XGB	0.7516	0.5204	0.7855	0.6261
LGBM	0.7550	0.5538	0.7721	0.6450
LR	0.7850	0.5723	0.7426	0.6464
CB	0.7608	0.5317	0.8097	0.6419
Vote	0.7956	0.5951	0.7131	0.6488

Observation:

- The soft-voting ensemble delivered the most balanced performance overall, achieving the highest **Accuracy** and **F1-score**.
- Random Forest attained the highest **Recall**, meaning it identified more actual churners, but it had the lowest **Precision**, indicating a higher number of false churns predictions.

7.2 Confusion Matrix

The following matrix shows how well the model separates **churners** from **non-churners.**

Random Forest Confusion Matrix:

	PREDICTED 0	PREDICTED 1
ACTUAL 0	**TN 314**	**FP 59**
ACTUAL 1	**FN 339**	**TP 697**

XGBOOST Confusion Matrix:

	PREDICTED 0	PREDICTED 1
ACTUAL 0	**TN 293**	**FP 80**
ACTUAL 1	**FN 270**	**TP 766**

LightGBM Confusion Matrix:

	PREDICTED 0	PREDICTED 1
ACTUAL 0	TN 288	FP 85
ACTUAL 1	FN 232	TP 804

Linear Regression Confusion Matrix:

	PREDICTED 0	PREDICTED 1
ACTUAL 0	TN 277	FP 96
ACTUAL 1	FN 207	TP 829

CatBoost Confusion Matrix:

	PREDICTED 0	PREDICTED 1
ACTUAL 0	TN 302	FP 71
ACTUAL 1	FN 266	TP 770

Soft Voting Confusion Matrix:

	PREDICTED 0	PREDICTED 1
ACTUAL 0	TN 266	FP 107
ACTUAL 1	FN 181	TP 855

Across all confusion matrices, **Soft Voting** catches the most churners (**highest TP=855, lowest FN=181**), while **Random Forest** is the most conservative (**highest TN=314, lowest FP=59**), showing the recall vs false-alarm trade-off.

7.3 Bootstrap Stability (95% Confidence Intervals)

Bootstrap resampling was used to estimate 95% confidence intervals (CI) for key metrics. Width (w) indicates stability: smaller widths imply more consistent performance across resamples.

Model	Accuracy (CI)	Precision (CI)	Recall (CI)	F1 (CI)
RF	0.732–0.779 (w=0.048)	0.496–0.558 (w=0.062)	0.732–0.818 (w=0.086)	0.596–0.657 (w=0.061)
XGB	0.767–0.809 (w=0.042)	0.548–0.619 (w=0.071)	0.660–0.753 (w=0.094)	0.606–0.672 (w=0.066)
LGBM	0.778–0.818 (w=0.040)	0.569–0.641 (w=0.072)	0.633–0.727 (w=0.094)	0.604–0.676 (w=0.072)
LR	0.774–0.813 (w=0.039)	0.568–0.645 (w=0.077)	0.582–0.678 (w=0.097)	0.581–0.655 (w=0.074)
CB	0.768–0.811 (w=0.043)	0.549–0.621 (w=0.072)	0.657–0.748 (w=0.091)	0.604–0.674 (w=0.070)
Vote	0.775–0.818 (w=0.043)	0.561–0.634 (w=0.073)	0.670–0.761 (w=0.091)	0.616–0.686 (w=0.070)

Bootstrap conclusion:

- The Voting ensemble maintains strong performance with competitive confidence intervals, indicating stablity. Random Forest provides the highest recall but is less precise, leading to more false alarms.

8. Error Analysis (Wrong Cases)

8.1 Definition

In this analysis, we focus on **unanimous wrong predictions**—cases where **all base models agreed on the same class**, but the prediction was still incorrect. These cases highlight **hard/borderline customers** that consistently confuse the models.

- **Unanimous FN :** All models predicted **0 (Not Churn)**, but true label was **1 (Churn)**.
- **Unanimous FP :** All models predicted **1 (Churn)**, but true label was **0 (Not Churn)**.

8.2 Observed Counts

- **Total test samples:** 1409
- **Unanimous FN:** 59
- **Unanimous FP:** 172
- **Total unanimous wrong:** 231 (**16.4%**)

Missed churn share: 4.2% | False alarm share: 12.2%

8.3 Interpretation

The unanimous false-alarm group is larger than the unanimous missed-churn group, suggesting that many non-churn customers have churn-like profiles (borderline cases).

9. Conclusion

Overall, the **Soft Voting ensemble** achieved the most **balanced performance** on the test set (**Accuracy = 0.7956, F1 = 0.6488**). While **Random Forest** captured more churners (higher recall), it also produced more false positives. The **bootstrapped confidence intervals** further support the stability and reliability of the ensemble results.

10. References

IBM Telco Customer Churn Dataset (benchmark dataset widely used for churn prediction research).